Exception handling

What is an Exception?

- The term *exception* is shorthand for the phrase "exceptional event.“
- **Definition:** An *exception* is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.

The try-catch statement

- When an exception occurs in a program, an object of java.lang. Throwable hierarchy representing the error state is thrown(executed)
- The thrown exception object has to be caught (handled) by the program code
- If it is not caught, the program execution is not permitted to continue.
- Put the code that could generate an exception within a try block as shown below:

```
……. // more statements
try{
/*  code to be tested  */
    }
 ……. // more statements
```

The try-catch statement cont.

- The statements enclosed in a try block are tested for any possible exceptions
- Every try block has an associated catch block which actually performs processing of the exception
- The keyword catch is followed by a pair of parenthesis which specifies an exception object that the catch block handles
- So the above code changes to the following:

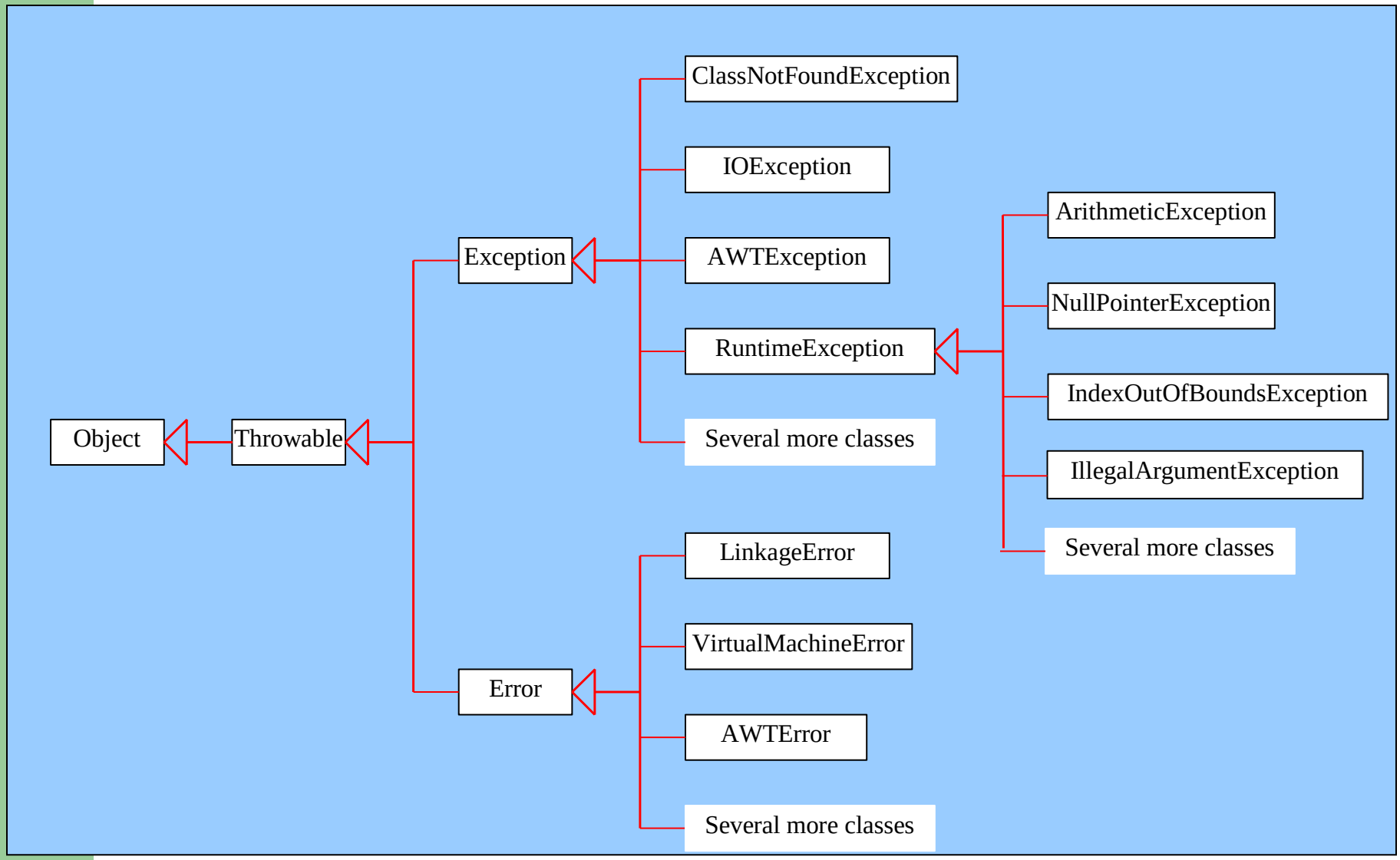
The try-catch statement cont.

```
……. // more statements
try{
            /*   code to be tested  */
    }
catch (Exception e)
{
   /*  exception handling code here */
}
……. // more statements
```

Throwable class

- java.lang.Throwable has two important subclasses, namely java.lang.Exception and java.lang.Error
- The java.lang.Exception and its subclasses handle the abnormalities in the program code that might arise during compile time or runtime
- java.lang.Error and its subclasses handle error conditions related to hardware, for example, memory not available
  - Linkage error, Virtual Machine error, etc…

Exception Classes



Throwable class cont.

- Hierarchy of java.lang.Error being associated with hardware,
- we as programmers will not use it directly, but often deal with java.lang.Exception and its subclasses
- One important subclass of java.lang.Exception is the class java.lang.RuntimeException.
- RuntimeException and its subclasses, as its name specifies, are responsible to handle scenarios where exceptions are thrown at runtime

Exception class

- catch block specifies an object of one of java.lang.Exception hierarchy
- It indicates the type of exception being handled by the catch block
- Of course, if catch block uses java.lang.Exception object, all runtime and compile time exceptions are handled since all other associated exceptions classes are derived from java.lang.Exception.
- This might make the code simple but does not allow us to handle different exception cases individually
- To do so, multiple catch blocks may be used with a single try block as follows:

Example

```
try
 {  //code to be tested
 }
catch (ArrayIndexOutOfBoundsException e1)
 {  //Handling of Array Indexing problems
 }
catch (ArrayStoreException e2)
 {  //Handling wrong type of object reference problems
 }
catch (Exception e3)
 {  //Handling all other exceptions  //should be at last
 }
```

Nesting try-catch statement

- The try-catch blocks can also be enclosed within another try block as shown below:

```
try{
     //code 1
          try{ //code2 }
          catch (ArithmeticException e)
          { // exception handling for arithmetic exceptions }
     //code 3
    }
catch (Exception e)
{ // exception handling }
```

The finally statement

- If an exception occurs in a try block, the remaining code (if any) within the same try block after the exception generating statement are skipped
- Also, if the associated catch block does not handle the exception, the execution control jumps to outer catch block (if any) or the program is terminated
- The code contained in a finally block is always executed irrespective of whether an exception is thrown or not.

```
try{ //code to be tested }
catch (Exception e)
{ //exception handling code here }
finally
{ // statements that should be executed
   compulsorily
}
```

The throws keyword

- If not want to include try-catch statement in program code
- prefer handling the exception elsewhere
- exception cannot be compiled until tested with try-catch
- throws keyword should be used in the method definition as follows:

```
void function1 () throws ArithmeticException
{ // method code }
```

The throws keyword cont.

- A method may also use multiple exception with the throws clause

```
void function1 () throws ArithmeticException,
   ArrayStateException
{
// method code
}
```

Use ‘,’ (comma)

The throw keyword

- all the examples discussed earlier, we saw that the exceptions are thrown either by the compiler or at runtime by JRE
- a programmer can make his program check for errors and decide to throw the exception by itself using throw

```
try{ //code to be tested }
catch (Exception e)
{ //exception handling code here
   throw e;  }
```

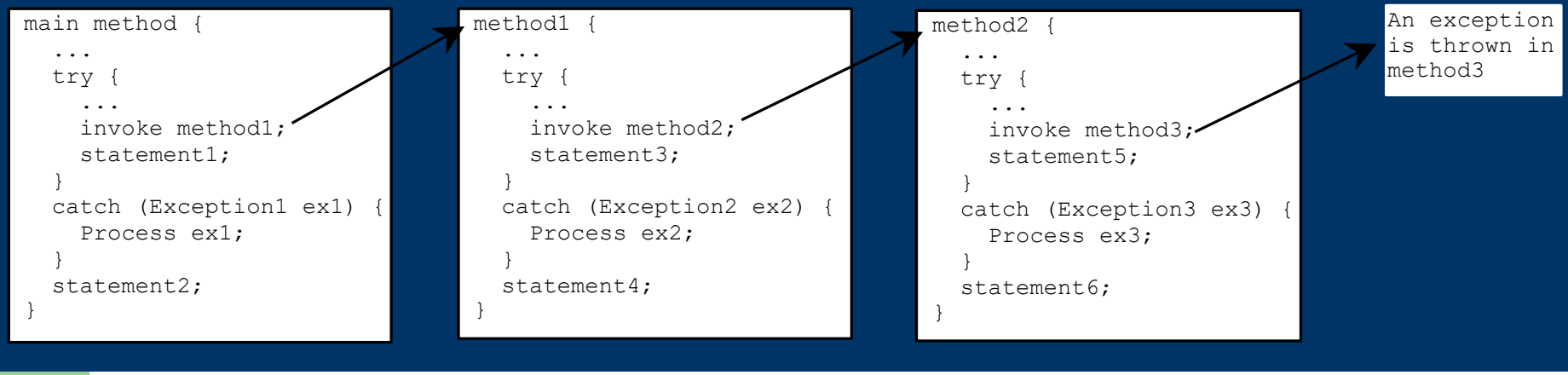
Throw in try block

```
try{
                //code to be tested
    throw new ArithmeticException();
   } catch (Exception e)
  {
   //exception handling code here
  }
```

Creating user defined exceptions

- exception objects used till were predefined
- developers can create their own exception classes
- We simply need to write a subclass of Exception and override the toString() method
- Whenever an exception object is printed, the respective objects toString method is invoked

Catching Exceptions



Catch or Declare Checked Exceptions

Java forces you to deal with checked exceptions. If a method declares a checked exception (i.e., an exception other than Error or RuntimeException), you must invoke it in a try-catch block or declare to throw the exception in the calling method. For example, suppose that method p1 invokes method p2 and p2 may throw a checked exception (e.g., IOException), you have to write the code as shown in (a) or (b).

```
void p1() {
  try {
    p2();
  }
  catch (IOException ex) {
    ...
  }
}
```

(a)

```
void p1() throws IOException {

  p2();

}
```

(b)

The `finally` Clause

```
try {
   statements;
}
catch(TheException ex) {
   handling ex;
}
finally {
   finalStatements;
}
```

Trace a Program Execution

Suppose no exceptions in the statements

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Trace a Program Execution

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The final block is always executed

Trace a Program Execution

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Next statement in the method is executed

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Suppose an exception of type Exception1 is thrown in statement2

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The exception is handled.

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The final block is always executed.

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The next statement in the method is now executed.

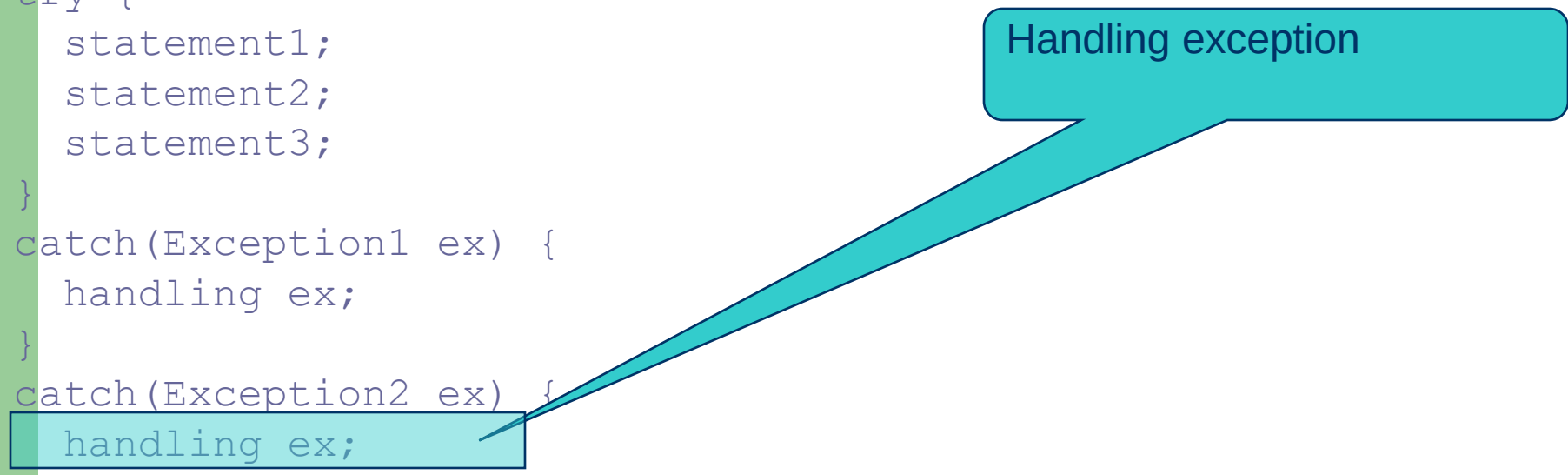
Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}
Next statement;
```

statement2 throws an exception of type Exception2.

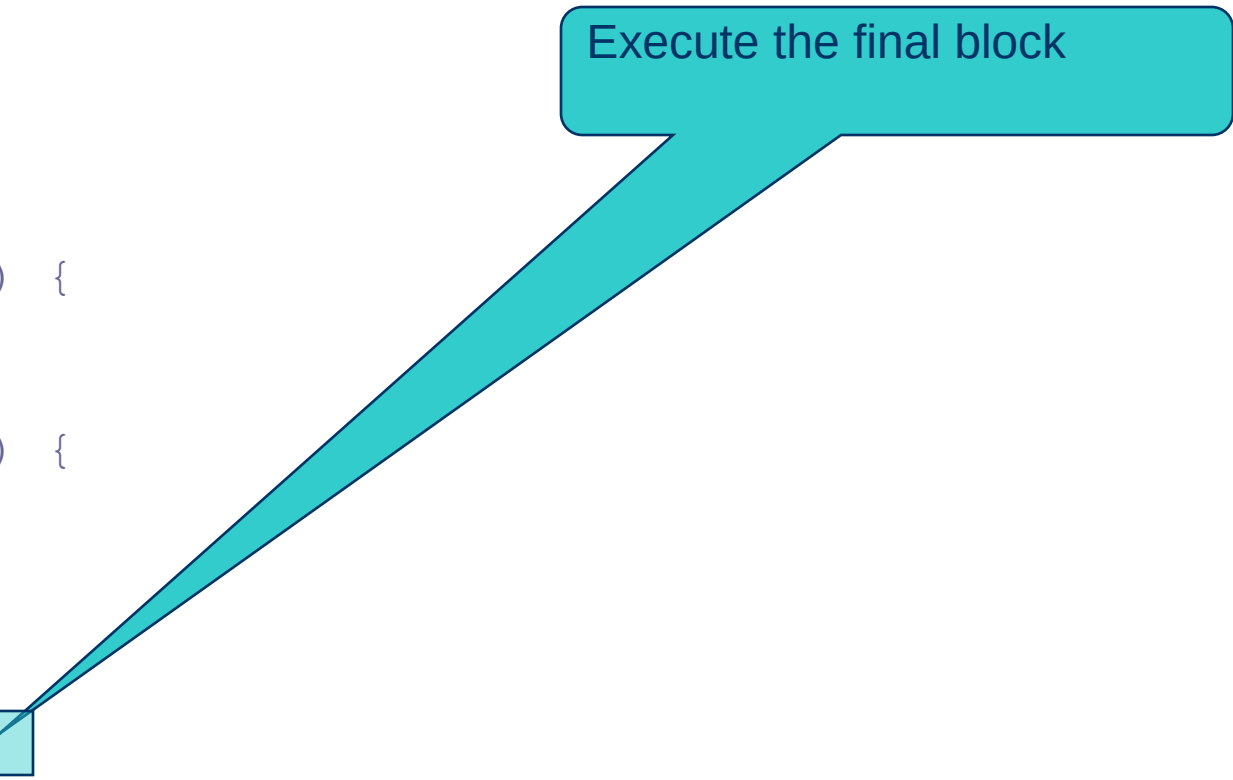
Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}
Next statement;
```



Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}
Next statement;
```



Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}
Next statement;
```

Rethrow the exception and control is transferred to the caller

When to Use Exceptions

When should you use the try-catch block in the code?

You should use it to deal with unexpected error conditions. Do not use it to deal with simple, expected situations. For example, the following code

```
try {
  System.out.println(refVar.toString());
}
catch (NullPointerException ex) {
  System.out.println("refVar is null");
}
```

When to Use Exceptions

is better to be replaced by

```
if (refVar != null)
  System.out.println(refVar.toString());
else
  System.out.println("refVar is null");
```